



UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA
FACULTAD DE INGENIERÍA EN SISTEMAS
ANÁLISIS DE SISTEMAS 2
OCTAVO CICLO 2026
SEDE: PORTALES

Entregable 1: Análisis, Rediseño Arquitectónico y Contrato API

Nombre: Victoriano Gómez Pérez

Carnet: 9989-23-18022

Sección: A

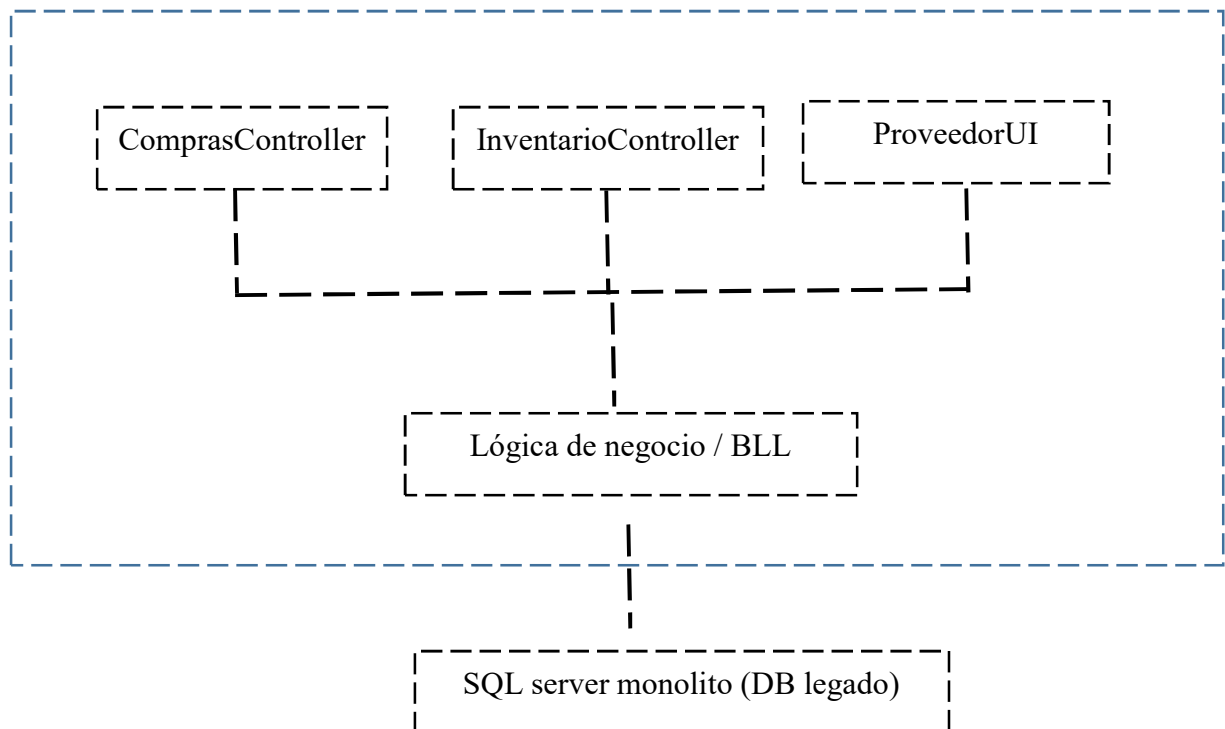
Guatemala 10 de Septiembre 2026

Componente 1: Informe de Ingeniería Inversa y Rediseño Arquitectónico

1. Contexto del Sistema Legado

- Estado actual: Monolito heredado desarrollado en C# .NET Framework 4.5 conectado directamente a una base de datos Microsoft SQL Server centralizada mediante llamadas a procedimientos almacenados dispersos.
- Puntos de dolor (cuellos de botella y acoplamiento)
 - Alto acoplamiento: Los controladores MVC contiene lógica de negocios, validaciones y acceso a datos acoplados sin separación conceptual.
 - Cuello de botella en inventario: Durante compras concurrentes masivas a proveedores, la base de datos colapsa por bloqueos de tablas.
 - Mantenibilidad: Dificultad para actualizar módulos individuales de compras sin romper dependencias de facturación o inventario.

2. Diagrama del sistema heredado(UML)



3. Matriz Comparativa de Arquitectura

Criterio	Sistema Monolítico Actual	Monolito Modular Propuesto (C# .NET 8)	Microservicios (EDA)
Escalabilidad	Baja (Escala verticalmente todo el sistema)	Media alta (Despliegue independiente por modulo)	Alta (Escala pods de C# de compras de forma independiente)
Mantenimiento	Compleja por código espagueti	Alta (Límites de contexto/Bounded Contexts claros)	Compleja (Requiere trazabilidad distribuida)
Riesgos	Colapso total por fallos en módulo compras	Bajo riesgos de migración progresivas	Complejidad de infraestructura y consistencia eventual
Decisión	Descartado	Seleccionado(Arquitectura limpia)	Descartado por costos operativos iniciales

4. Justificación Técnica (Atributos de Calidad)

- Escalabilidad: Separacion de los endpoints del módulo de compras mediante C# ASP.NET Core Web API, permitiendo uso de Redis como cache de lectura.
- Mantenibilidad: Aplicación del patrón arquitectura limpia en visual studio (Core, Aplicación, infraestructura, API).
- Disponibilidad: Desacoplamiento de llamadas sincronicas en lectura mediante caché Redis (Cache-Aside).
- Seguridad: Autenticacion estandarizada con JWT (JSON Web Tokens) para proteger las operaciones de compra.

Componente 2: Especificación y Contrato OpenAPI 3.0 (Swagger)

openapi: 3.0.3

info:

title: API de Gestión de Compras - Farmacias Galeno

description: Servicio Web API en C# / ASP.NET Core para la gestión de órdenes de compra, proveedores y recepciones de inventario.

version: 1.0.0

servers:

- url: https://api.farmaciasgaleno.com/v1

description: Servidor Producción

paths:

/compras/ordenes:

get:

summary: Listar órdenes de compra

security:

- bearerAuth: []

parameters:

- name: estado

in: query

schema:

type: string

enum: [Pendiente, Aprobada, Recibida, Cancelada]

responses:

'200':

description: Lista de órdenes obtenida correctamente

content:

application/json:

schema:

type: array

items:

\$ref: '#/components/schemas/OrdenCompraResponse'

'401':

description: No autorizado

'500':

description: Error interno del servidor

post:

summary: Crear nueva orden de compra

security:

- bearerAuth: []

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/CrearOrdenCompraRequest'

responses:
'201':
 description: Orden de compra creada con éxito
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/OrdenCompraResponse'
'400':
 description: Petición inválida o datos de entrada incorrectos

/compras/ordenes/{id}:

get:
 summary: Obtener detalle de una orden de compra por ID
 security:
 - bearerAuth: []
 parameters:
 - name: id
 in: path
 required: true
 schema:
 type: integer
 responses:
 '200':
 description: Detalle de la orden de compra
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/OrdenCompraResponse'
 '404':
 description: Orden de compra no encontrada

/compras/ordenes/{id}/recepcion:

post:
 summary: Registrar recepción de mercancía de una orden
 security:
 - bearerAuth: []
 parameters:
 - name: id
 in: path
 required: true
 schema:
 type: integer
 requestBody:
 required: true
 content:
 application/json:
 schema:

\$ref: '#/components/schemas/RecepcionOrdenRequest'

responses:

'200':

description: Recepción procesada e inventario actualizado

'400':

description: Error en los datos de recepción

components:

securitySchemes:

bearerAuth:

type: http

scheme: bearer

bearerFormat: JWT

schemas:

CrearOrdenCompraRequest:

type: object

required:

- proveedorId
- items

properties:

proveedorId:

type: integer

example: 10

observaciones:

type: string

example: "Entrega urgente medicamentos fríos"

items:

type: array

items:

\$ref: '#/components/schemas/DetalleOrdenRequest'

DetalleOrdenRequest:

type: object

required:

- productoId
- cantidad
- precioCosto

properties:

productoId:

type: integer

example: 501

cantidad:

type: integer

example: 100

precioCosto:

type: number

format: decimal
example: 12.50

OrdenCompraResponse:

type: object
properties:
 id:
 type: integer
 example: 1205
 proveedorId:
 type: integer
 example: 10
 fechaEmision:
 type: string
 format: date-time
 estado:
 type: string
 example: "Pendiente"
 total:
 type: number
 format: decimal
 example: 1250.00

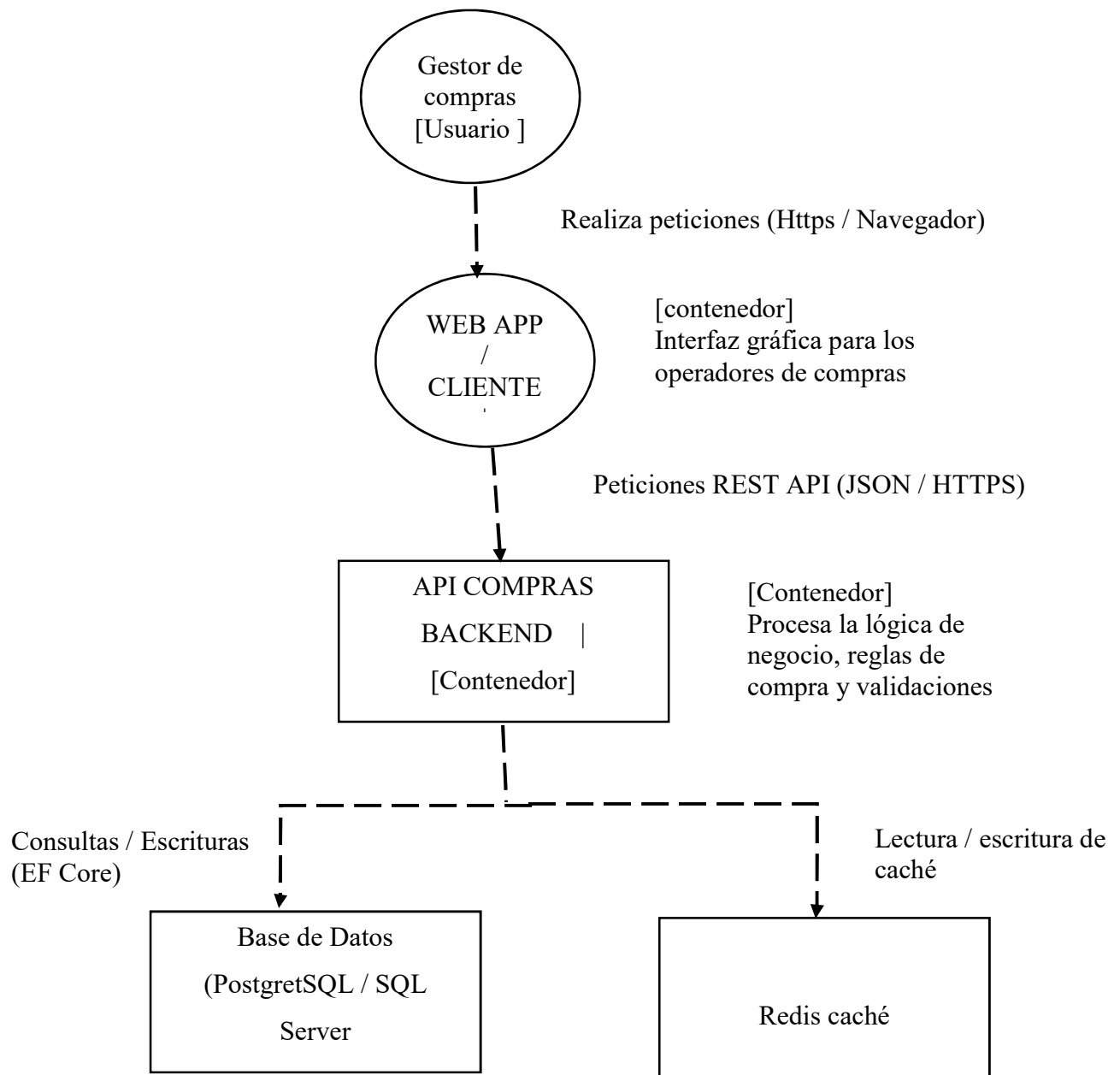
RecepcionOrdenRequest:

type: object
required:
 - numeroFacturaProveedor
 - detalles
properties:
 numeroFacturaProveedor:
 type: string
 example: "FACT-2026-991"
 detalles:
 type: array
 items:
 type: object
 properties:
 productoId:
 type: integer
 example: 501
 cantidadRecibida:
 type: integer
 example: 100
 fechaVencimiento:
 type: string
 format: date
 example: "2028-06-30"

lote:
type: string
example: "LOT-8819A"

Componente 3: Planos Técnicos bajo el Modelo C4

Nivel 1: Diagrama de Contexto



[Contenedores de Persistencia y caché]

- DB Almacena órdenes, catálogo y proveedores
- Redis: Almacena catálogo en caché para lecturas rápidas

Detalles de la interacción entre bloques:

Gestor de Compras (Usuario): Interactúa con la aplicación frontend desde su navegador Web.

Web App / Cliente (React/Blazor): Envía las solicitudes de compras mediante peticiones RESTful tipadas (GET, POST, PUT) utilizando objetos JSON.

API Compras Backend (C# .NET 8): Recibe las solicitudes, ejecuta el pipeline de autenticación (JWT) y decide si buscar información en la memoria de Redis o consultar la base de datos principal.

Redis Caché: Proporciona un tiempo de respuesta ultrarrápido (Cache-Aside) para listas recurrentes como catálogos de medicamentos o proveedores activos.

Base de Datos Principal: Mantiene la integridad referencial y almacena el histórico permanente de las órdenes de compra y recepción de inventario.

Nivel 2: Diagrama de Contenedores

Personas / actores

- Gestor de compras: Usuario crea y aprueba órdenes de compra.
- Encargado de bodega: Usuario que registra las recepciones físicas de productos

Contenedores de Aplicación / Servicios

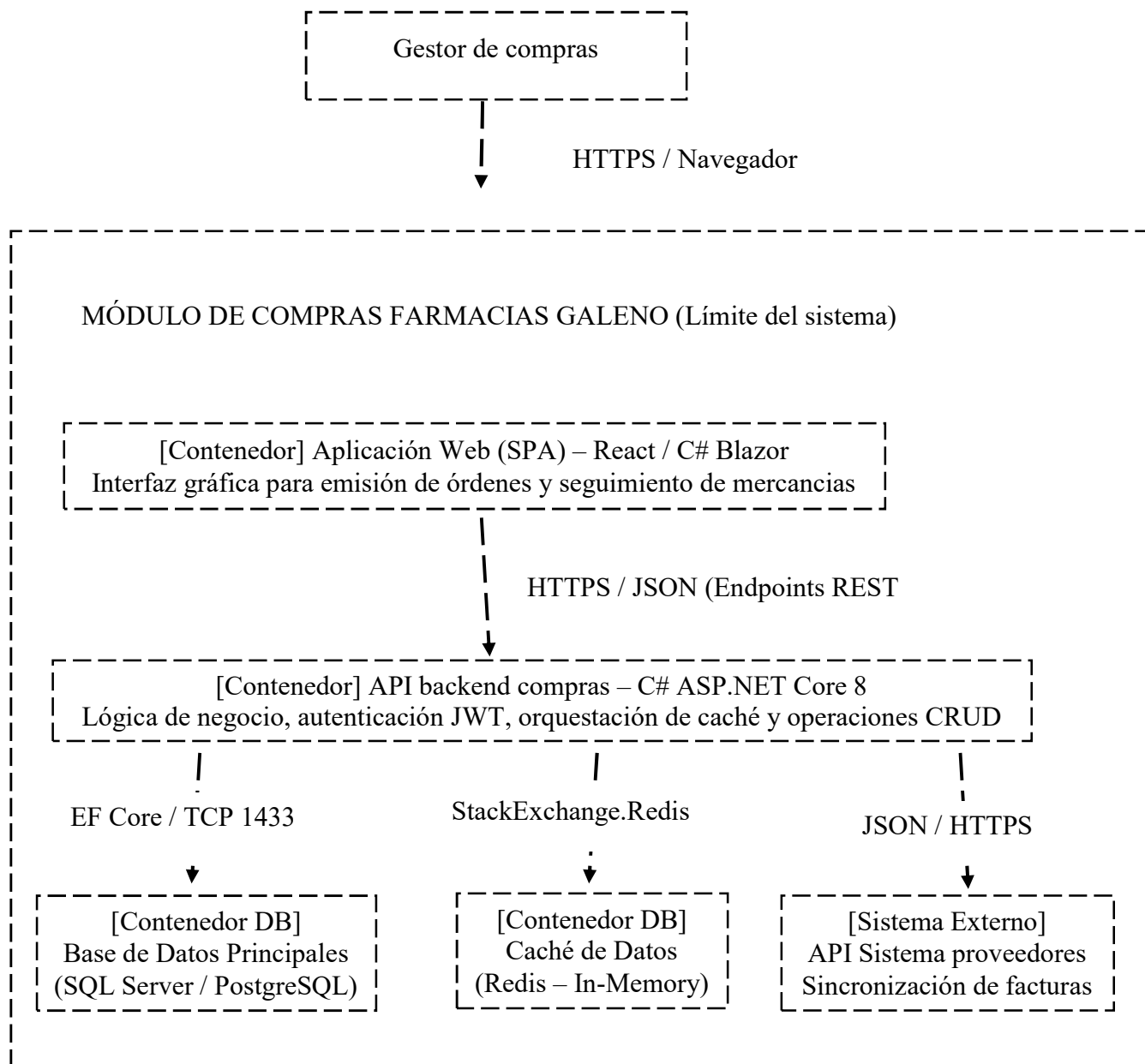
- Web aplicación (Frontend): Desarrollo SPA en React / Blazor. Se ejecuta en el navegador del usuario y gestiona la interfaz gráfica.
- API compras Backend (Servicio C# .NET 8): Servicio ASP.NET Core Web API que expone los endpoints RESTful, ejecuta las reglas de negocio de compras y procesa los datos.

Contenedores de almacenamiento:

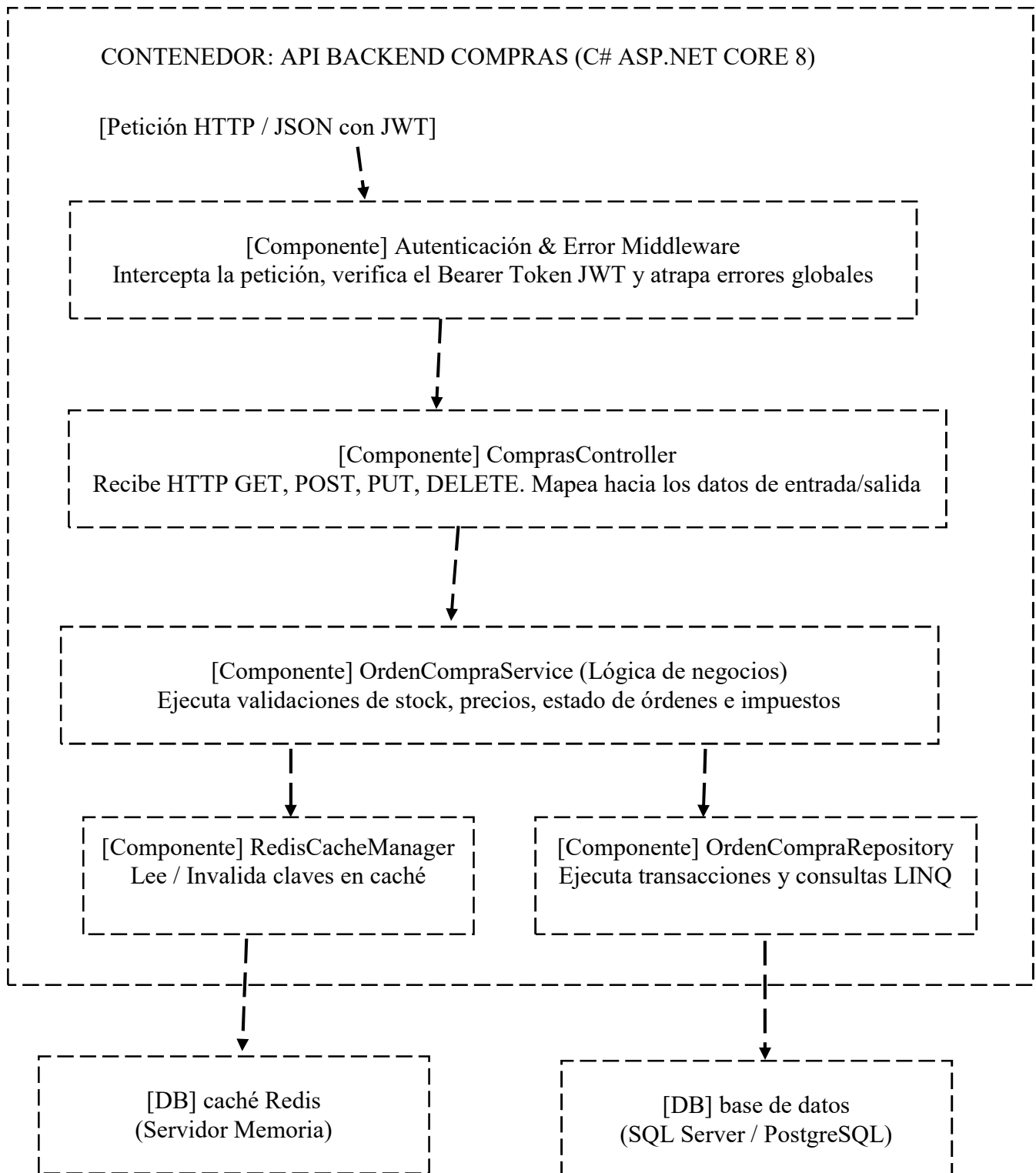
- Base de datos relacional: Microsoft SQL Server o PostgreSQL, encargada de la persistencia transaccional (tablas de proveedores, productos, órdenes y recepción).
- Caché Redis: Almacenamiento en memoria para consultas rápidas (cache-Aside) de listas que no mutan frecuentemente (catálogo de medicamentos, proveedores activos).
-

Sistemas Externos:

- Sistemas proveedor exterior: API REST/SOAP externa para consulta de facturas electrónicas y estado de despachos.



Nivel 3: Diagrama de Componentes



Componente 4: Diseño de Persistencia de Datos y Estrategia de Caché

Diagrama de Persistencia:

Script DDL de Base de Datos:

--Creación de Tablas

```
CREATE TABLE Proveedor (  
    ID_Proveedor INT IDENTITY (1,1) PRIMARY KEY,  
    NIT NVARCHAR (20) NOT NULL UNIQUE,  
    DIRECCIÓN NVARCHAR (150) NOT NULL,  
    TELEFONO NVARCHAR (20),  
    Estado BIT DEFAULT 1  
);
```

```
CREATE TABLE Producto (  
    ID_Producto INT IDENTITY (1,1) PRIMARY KEY,  
    Nombre_Producto NVARCHAR(150) NOT NULL,  
    RequiereReceta BIT DEFAULT 0,  
    PrecioCostoUnitario DECIMAL (18,2) NOT NULL  
);
```

```
CREATE TABLE OrdenCompra (  
    ID_OrdenCompra INT IDENTITY (1,1) PRIMARY KEY,  
    ID_Proveedor INT NOT NULL,  
    FechaEmision DATETIME DEFAULT GETDATE(),  
    Estado NVARCHAR (20) CHECK (Estado IN ('Pendiente', 'Aprobada', 'Recibida', 'Cancelada')),  
    Total DECIMAL (18,2) NOT NULL,  
    Observaciones NVARCHAR (500),  
    CONSTRAINT FK_OrdenCompra_Proveedor FOREIGN KEY (ID_Proveedor) REFERENCES  
Proveedor (ID_Proveedor)  
);
```

```

CREATE TABLE DetalleOrdenCompra (
    ID_Detalle INT IDENTITY (1, 1) PRIMARY KEY,
    ID_OrdenCompra INT NOT NULL,
    ID_Producto INT NOT NULL,
    CantidadSolicitada INT NOT NULL,
    PrecioCostoUnitario DECIMAL (18,2) NOT NULL,
    CONSTRAINT FK_Detalle_Orden FOREIGN KEY (ID_OrdenCompra) REFERENCES
OrdenCompra (ID_OrdenCompra),
    CONSTRAINT FK_Detalle_Producto FOREIGN KEY (ID_Producto) REFERENCES
Producto(ID_Producto)
);

```

Estrategia de Caché (Redis):

- Patrón Aplicado: Cache-Aside.
- Llaves de Caché (Cache Keys) y TTL
 - :compras:proveedores:activos | TTL: 12 horas.
 - compras:ordenes:{id} | TTL: 30 minutos.
- Estrategia de Invalidador: Al invocar operaciones de mutación (POST /compras/ordenes/{id}/repcion o actualización de ordenes), el servicio RedisCacheManager invalida explícitamente las claves compras:ordenes:{id} y el listado general de órdenes para evitar lectura de datos caducos (Stale Data).

Estructura Final del Repositorio